

Dynamic Wallrun

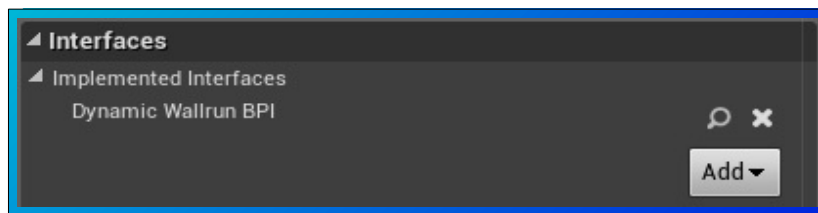
Hello, and thank you for your purchase! This documentation covers the following topics:

- Merging Into Your Project
- Integration with ALS
- Customizing The System

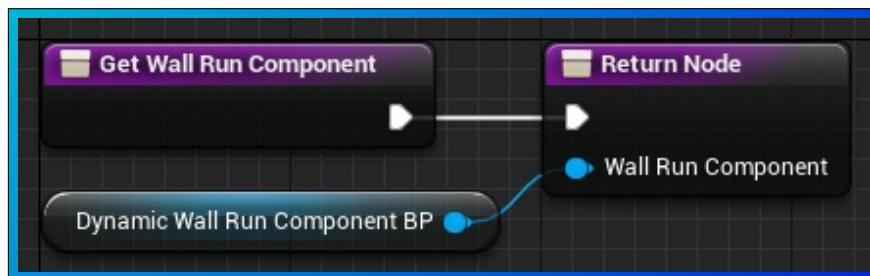
Merging Into Your Project

The following steps will guide you through migrating this system into your project and merging it into an existing character.

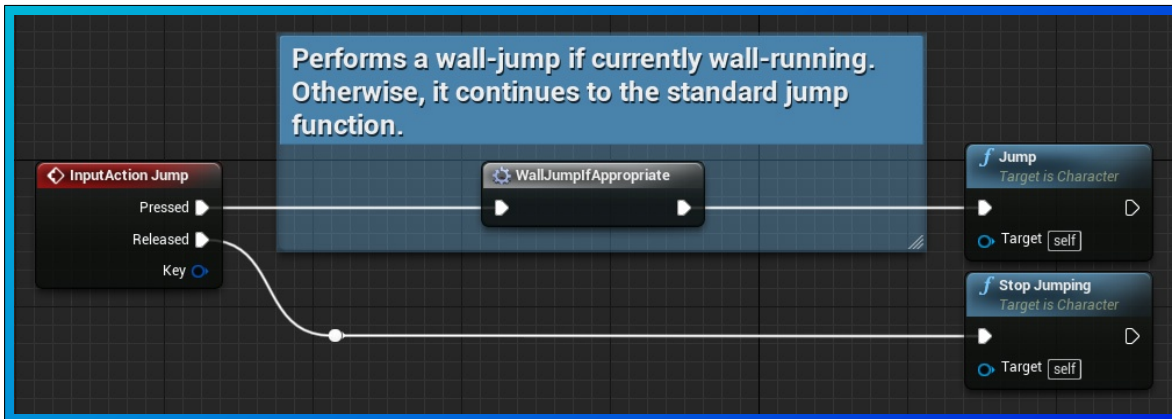
1. Open the provided **Dynamic Wallrun** project.
2. Right click on the **DynamicWallRun** folder in the content browser and select **Migrate**. Select the **Content** folder of the project you want to migrate to.
3. Close the **Dynamic Wallrun** project and open your project.
4. Open your existing character BP and add the **DynamicWallRunComponent_BP** to it.
5. In your character BP's class settings, add the **DynamicWallrun_BPI** interface.



6. On the left side of the character BP, double-click on the **GetWallRunComponent** function in the **Interfaces** category. Click on your **DynamicWallRunComponent_BP**, drag a reference into the graph, and plug it into the **Wall Run Component** input on the **Return Node**.



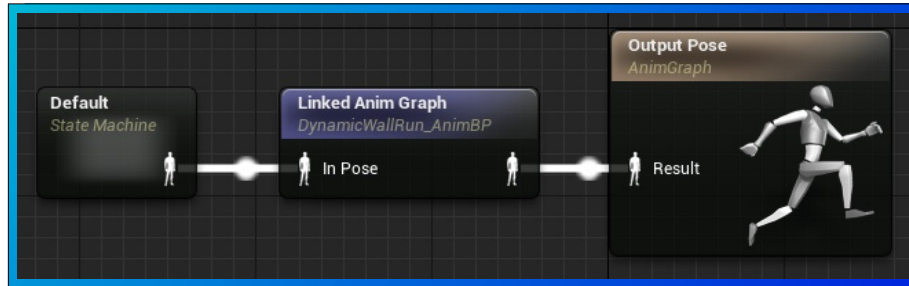
7. Locate your character's jump input action. Add the **WallJumpIfAppropriate** macro by right-clicking in any open space on the graph and typing its name until it appears in the list. Hook the macro in so that it is the first thing triggered by the input event, then connect its output to your normal jump logic.



8. Compile and close the character BP.
9. We will now retarget the AnimBP. *The steps here are for UE5.4 – earlier engine versions may have a different retarget process.* Right-click on the included **DynamicWallRun_AnimBP** (DynamicWallRun/Animations) and select **Retarget Animations**.
10. Choose your character's mesh for the **Target Skeletal Mesh**. Make sure that **DynamicWallRun_AnimBP** is selected in the list of animations, then click Export Animations.
11. Choose a location for the retargeted animations, then click **Export**. In the dialog that appears, ensure that the **Include Referenced Assets** option is enabled, then click **Export** again.
12. Unfortunately, there is a bug in UE5 that causes additive settings to be removed from animations during retargeting. There are 4 wall-run animations that require additive settings, so you may have to open each of these and restore their additive settings manually:
 - All 4 of these need to have their **Additive Anim Type** set to **Local Space**
 - All 4 need to have their **Base Pose Type** set to **Selected Animation Frame**
 - **WallRun_HandCorrectionAdditive_L** – Set its Base Pose Animation to your retargeted **WallRun_Ascending_L**
 - **WallRun_HandCorrectionAdditive_R** – Set its Base Pose Animation to your retargeted **WallRun_Ascending_R**
 - **WallRun_OutsideCornerAdditive_L** – Set its Base Pose Animation to your retargeted **WallRun_L**
 - **WallRun_OutsideCornerAdditive_R** – Set its Base Pose Animation to your retargeted **WallRun_R**

I apologize for this inconvenience, but until Epic fixes this engine bug, making these corrections is required.

13. Open the AnimBP you're using for your character. In the main AnimGraph, create a **Linked Anim Graph** node (also called **Sub Anim Instance** in older engine versions). With this node selected, locate its **Details** panel and set the **Instance Class** to your retargeted **DynamicWallRun_AnimBP**.
14. Plug your existing state machine / final pose result into the **In Pose** on the **Linked Anim Graph**, then connect its output to the **Output Pose**.

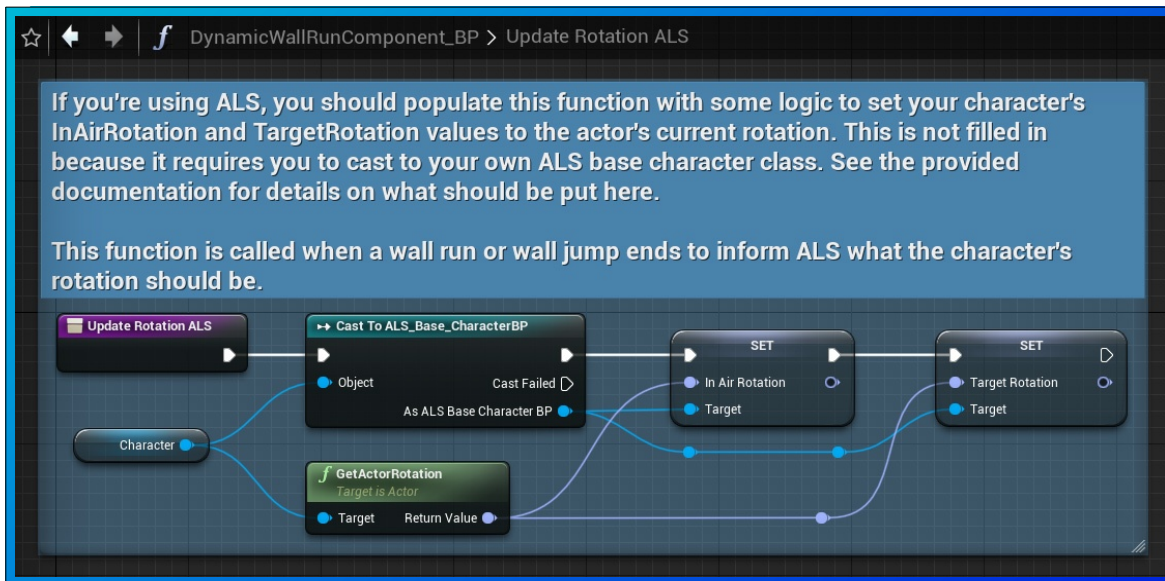


15. Compile and save the AnimBP, and you're done!
16. I recommend testing the wall-run now to see how it looks. If you have a thin character capsule, your character mesh may clip into the wall while wall-running. This can be corrected by using the wall run component's **CharacterOffset** parameter to push the character further away from the wall.

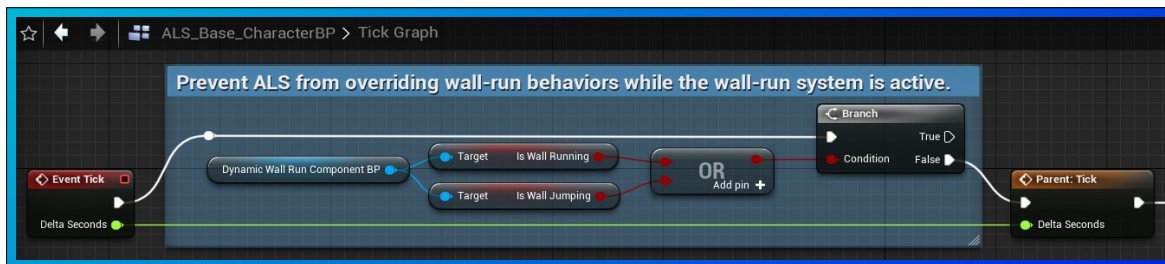
Integration With ALS

If you're using Advanced Locomotion System, there are a couple extra steps to take for best results. After completing the previous section, I recommend doing the following:

1. When a wall run ends, we want to inform ALS of the desired character rotation. Open the **DynamicWallRunComponent_BP** (DynamicWallRun\Blueprints.Components) and open its **UpdateRotationALS** function. Cast the character to your ALS character BP and set its **InAirRotation** and **TargetRotation** variables to the character's current rotation.



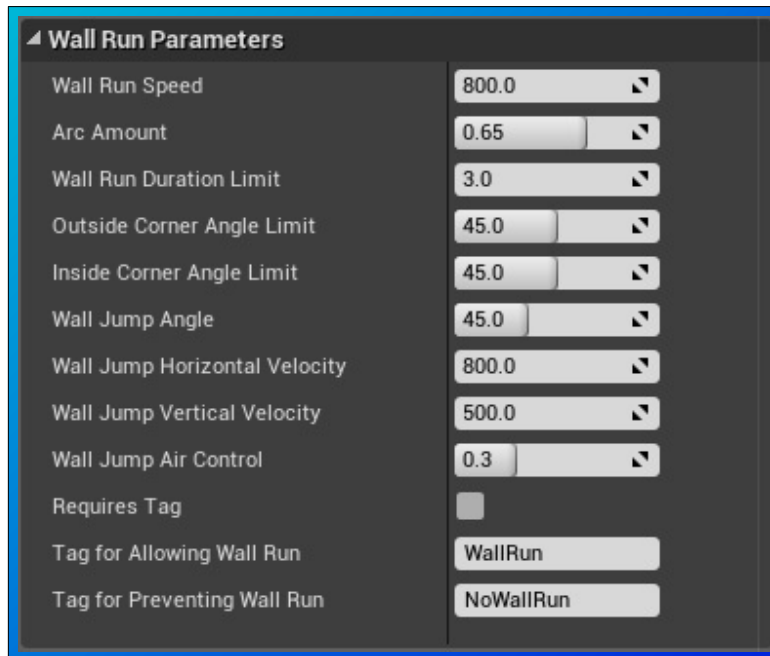
2. Next, we want to prevent ALS from performing actions that will interrupt/conflict with what the Wall Run system is trying to do. We can do this by opening the ALS character BP, then placing a branch at the beginning of its **Event Tick** node inside the **Tick Graph**. The branch should check if **IsWallRunning** or **IsWallJumping** are true, and only proceed if they are both false:



3. Compile and save the BPs, and you're done!

Customizing The System

The **DynamicWallRunComponent** has many parameters that allow you to customize how your wall-run behaves. These properties can be accessed by selecting the component in your character BP, then locating the **Wall Run Parameters** category in the details panel.



Wall Run Parameters

Wall Run Speed

How fast the wall run should be.

Arc Amount

How much arc the wall run should have. 1 = Significant arc, 0 = No arc (100% horizontal).

Wall Run Duration Limit

Number of seconds that a wall run can be active before it automatically ends. Set to 0 for infinite duration.

Outside Corner Angle Limit

How sharp of an outside corner the player can turn while wall running, in degrees. WARNING: If the WallRunSpeed is set too high, it can hinder the character's ability to turn very sharp (~90 degree) outside corners!

Inside Corner Angle Limit

How sharp of an inside corner the player can turn while wall running, in degrees.

Wall Jump Angle

The angle at which the character will be launched away from the wall when wall-jumping. 90 degrees = perpendicular to the wall.

Wall Jump Horizontal Velocity

The horizontal velocity at which the character will be launched from the wall when wall-jumping.

Wall Jump Vertical Velocity

The vertical velocity at which the character will be launched from the wall when wall-jumping.

Wall Jump Air Control

How much control the player has over their character's movement while in the air after wall jumping. 0 = no control (player can't turn or move while falling), 1 = full control.

CharacterOffset

If your character's hands/feet are clipping through the wall while wall-running, this can help correct that. Positive values push the character away from the wall, negative values pull the character towards the wall. Value is distance in standard Unreal units (cm).

Requires Tag

If enabled, the player will only be allowed to wall run on objects that have the actor tag specified by the **TagForAllowingWallRun** parameter. NOTE: Brush geometry does not support actor tags, so if this option is enabled, wall running will not work on any brush geometry - you must use meshes!

Tag for Allowing Wall Run

Actor tag that can be used to specify which actors the player can wall-run on. This is only used if the **RequiresTag** parameter is set to True. NOTE: This tag will have no effect on brush geometry because brushes do not support actor tags.

Tag for Preventing Wall Run

Actor tag that can be placed on actors that you do not want to be wall-run on. NOTE: This tag will have no effect on brush geometry because brushes do not support actor tags.